



Universidad
de Alcalá

fair-publish

Policy-enforced FAIR metadata validation and publishing pipeline

Technical overview & demo

M.-A. Sicilia · A. Ballesteros-Rodríguez · E. García-Barriocanal · M.
Mora-Cantalops · A. Nogales

University of Alcalá | IRyCIS

The metadata publication problem

In a research organisation, a dataset record must be:

- ▶ Described in a **Data Management Plan** (DMP)
- ▶ Validated against **institutional policy**
- ▶ Published to **one or more FAIR repositories**
- ▶ **Versioned** when the dataset evolves

Today this is done **manually, inconsistently, and late**

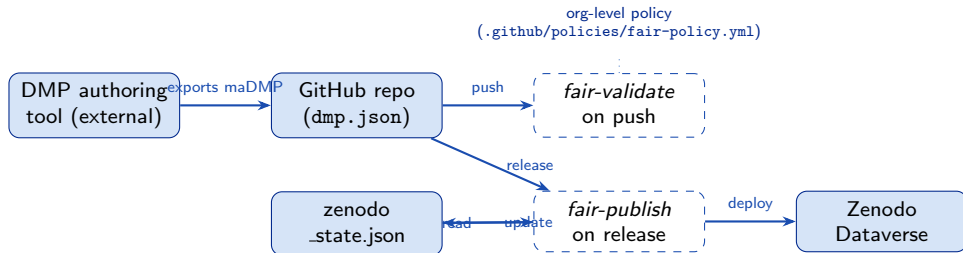
The CI/CD analogy

Software engineering solved the same problem:

source of truth (code) → **automated tests** (policy) → **release** (repository)

fair-publish applies this pattern to **FAIR metadata publication**

System architecture



Two-workflow design

Workflow 1: `fair-validate.yml`

Trigger: push / pull request to main

- ▶ Structural validation via `madmpy`
- ▶ Policy rule checks (per dataset)
- ▶ Blocks merge if any error-level rule fails
- ▶ Writes GHA step summary with rule-by-rule table

Governance requirement

Branch protection rules must require this workflow to pass — otherwise a PI can release without validation

Workflow 2: `fair-publish.yml`

Trigger: GitHub release published

- ▶ Publishes each dataset in the DMP to Zenodo
- ▶ New dataset → new Zenodo record
- ▶ Known dataset → new version (via `zenodo_state.json`)
- ▶ Commits updated state file back to repo
- ▶ Creates `published/<tag>` git tag

Built-in FAIR rules

Rules operate on (DMP, Dataset) pairs from madmpy objects.

Built-in rules:

- ▶ Creator ORCID present *F1, A1*
- ▶ License present & recognized *R1.1*
- ▶ At least one keyword *F2*
- ▶ Description $\geq$ N chars *F2, R1*
- ▶ Access rights stated *A1*
- ▶ Dataset identifier present *F1*

Each rule returns:

passed, severity, field, message,
fair

Listing 1: Rule protocol

```
class Rule(Protocol):  
    name: str  
    severity: str # "error" / "warning"  
    fair: str  
  
    def __call__(  
        self, dmp: Any, dataset: Any  
    ) -> RuleResult: ...
```

error → blocks publishing

warning → reported only

Institutions add domain-specific rules without modifying the core package.

Listing 2: Example: SNOMED CT rule

```
# myorg_rules/biomedical.py
from fair_publish.rules import RuleResult

class SnomedCtKeyword:
    name = "SNOMED CT keyword present"
    severity = "error"
    fair = "I1, R1.3"

    def __call__(self, dmp, dataset):
        kws = getattr(dataset, "keyword", []) or []
        ok = any("snomed" in k.lower()
                 for k in kws)
        return RuleResult(
            passed=ok, rule=self.name,
            severity=self.severity,
            field="dataset.keyword",
            fair=self.fair,
            message="" if ok else
                "No SNOMED CT keyword found.",
        )
```

Listing 3: Register in policy.yml

```
rules:
  - name: Creator ORCID present
    severity: error
  - name: License present and recognized
    severity: error
  - name: At least one keyword
    severity: error
  - name: Description length
    severity: warning
    config:
      min_chars: 150

extra_rules:
  - module: myorg_rules.biomedical
    class: SnomedCtKeyword
    severity: error

targets:
  - zenodo
```

maDMP as source of truth

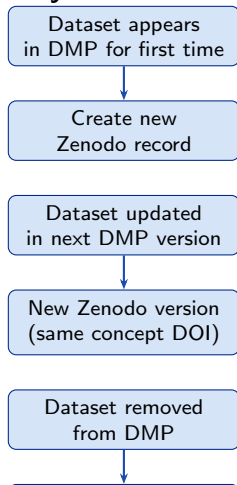
One DMP per project, one or more datasets per DMP.

The DMP is authored in an **external researcher-friendly tool** and exported as RDA maDMP JSON.

Key fields used by the pipeline:

- ▶ `dmp.contact / contributor` → creators
- ▶ `dataset[].title, description, keyword`
- ▶ `dataset[].distribution[].license` → SPDX id

Dataset lifecycle across DMP



State tracking: zenodo_state.json

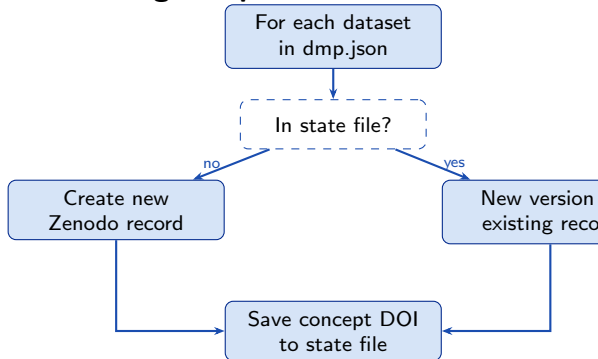
Committed alongside `dmp.json` in the project repository.
Maps `dataset_id.identifier` → Zenodo concept DOI.

Listing 4: After first release

```
{  
  "https://doi.org/10.5281/zenodo.AAAA":  
    "10.5281/zenodo.AAAA",  
  "https://github.com/myorg/proj/ds-B":  
    "10.5281/zenodo.BBBB"  
}
```

Auto-committed by the pipeline bot after each successful release. No manual editing required.

Decision logic at publish time:



Adopting fair-publish — three steps

Step 1 — Org-level setup (once)

- ▶ Add workflows to ORG/.github
- ▶ Commit fair-policy.yml
- ▶ Store ZENODO_TOKEN as org secret

Step 2 — Per project (minimal)

```
# .github/workflows/publish.yml
on:
  release:
    types: [published]
jobs:
  publish:
    uses: MY-ORG/.github/
      .github/workflows/
      fair-publish.yml@v1
    with:
      dmp_path: dmp.json
    secrets:
      zenodo_token:
        ${ secrets.ZENODO_TOKEN }
```

Step 3 — Branch protection (governance)

- ▶ Require *Validate maDMP* status check
- ▶ Apply via org-level Ruleset → propagates to all project repos automatically

What the PI sees

1. Edit DMP in institutional tool
2. Tool pushes dmp.json to GitHub
3. Validation result visible in pull

Running the pipeline locally

`simulate_release.py` reproduces the full PI release cycle against the Zenodo sandbox — no GitHub repository or production credentials needed.

```
# Install the package
pip install fair-publish

# Validate only (fast feedback)
fair-publish validate example/dmp.json --policy example/policy.yml

# Full simulation: validate + publish to Zenodo sandbox
python simulate_release.py \
  --dmp example/dmp.json \
  --policy example/policy.yml \
  --release-tag v1.0.0

# Output:
# == Step 1: Validate maDMP ==
# DMP: Longitudinal cohort study of metabolic biomarkers
# Datasets: 1 Errors: 0 Warnings: 0
# PASSED
#
# == Step 2: Publish datasets ==
# [created] Metabolic biomarker dataset ... -> 10.5281/zenodo.XXXX
#
# State written to: example/zenodo_state.json
```

Validation report in GHA

The step summary written to the GitHub Actions UI after each push:

```
## FAIR Validation Report
  PASSED  0 errors, 1 warning

### Dataset: Metabolic biomarker dataset  cohort wave 1-5
| Rule | FAIR | Status | Message |
|-----|-----|-----|-----|
| Creator ORCID present | F1, A1 | | OK |
| License present and recognized | R1.1 | | OK |
| At least one keyword | F2 | | OK |
| Description length | F2, R1 | | 198 chars; min 150 |
| Access rights stated | A1 | | OK |
| Dataset identifier present | F1 | | OK |
```

What fair-publish provides:

- ▶ Pluggable rule protocol — built-in FAIR rules + institution domain rules
- ▶ YAML policy config at org level, inherited by all projects
- ▶ Per-dataset lifecycle tracking (`zenodo_state.json`)
- ▶ Two-workflow GHA design: validate on push, deploy on release
- ▶ Git tags for publication status history
- ▶ CLI for local validation and sandbox simulation

Assumptions & scope

- ▶ DMP authoring tool is external
- ▶ Repos owned by institution
- ▶ Branch protection configured org-wide
- ▶ Metadata only (no data file upload)

Code

```
https://github.com/irycis/fair-publish  
pip install fair-publish
```



Universidad
de Alcalá

Questions?

`msicilia@uah.es`

`https://github.com/irycis/fair-publish`